LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [04] – [Enkapsulasi]



I Kadek Rio Adi Pranata Kusuma

103132400029

S1-SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Modul praktikum Pemrograman Berorientasi Objek ini membahas konsep dasar enkapsulasi dalam Java yang berfokus pada tiga komponen utama, yaitu *Modifier*, *Constructor*, dan *Package*. *Modifier* berfungsi untuk mengatur tingkat akses serta sifat dari suatu kelas, metode, atau variabel (seperti pengunaan enkapsulasi data *hiding* melalui atribut *private* dan metode *public*).

Selain itu, *constructor* digunakan sebagai metode khusus untuk menginisialisasi dan menciptakan objek baru—yang dapat dibuat dengan berbagai parameter (*overloading*)—sementara *package* berperan penting dalam mengorganisasi serta mengelompokkan kelas-kelas dalam direktori tertentu guna menghindari konflik penamaan.

B. Guided

1. [Constructor – DemoManusia]

```
import com.rio.projectmodul04.Guided.manusia.Manusia;
public class DemoManusia {
    public static void main(String[] args) {

        Manusia arrMns[] = new Manusia[3];

        // Constructor pertama
        Manusia objMns1 = new Manusia();

        // Constructor kedua
        Manusia objMns2 = new Manusia("John");

        // Constructor ketiga
        Manusia objMns3 = new Manusia("Bajuri", 44);

        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;

        for (int i = 0; i < 3; i++) {
            System.out.println("Nama: " + arrMns[i].getNama());
            System.out.println("Umur: " + arrMns[i].getUmur());
        }
    }
}
```

[Constructor – Manusia]

```
package com.rio.projectmodul04.Guided.manusia;

public class Manusia {
    private String nama;
    private int umur;

    // Constructor pertama
    public Manusia() {
    }

    // Constructor kedua
    public Manusia(String nama) {
        this.nama = nama;
    }

    // Constructor ketiga
    public Manusia(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }

    // Setter
    public void setNama(String a) {
        nama = a;
    }

    public void setUmur(int a) {
        umur = a;
    }

    // Getter
    public String getNama() {
        return nama;
    }

    public int getUmur() {
        return umur;
    }
}
```

Output :

```
-----------------------< com.Rio:projectmodul04 >-----------------------
Building projectmodul04 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ projectmodul04 ---
skip non existing resourceDirectory C:\Semester 5\pbo file\PBO\templat repo\praktikum-pbo-ps1ds-05-reg-01\m

--- compiler:3.15.0:compile (default-compile) @ projectmodul04 ---
Recompiling the module because of changed source code.
Compiling 3 source files with javac [debug release 26] to target\classes

--- exec:3.6.3:exec (default-cli) @ projectmodul04 ---
Nama: null
Umur: 0
Nama: John
Umur: 0
Nama: Bajuri
Umur: 44
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.166 s
Finished at: 2026-10-12T19:11:03+07:00
------------------------------------------------------------------------
```

2. [Package - hargaPulsa]

```
public class hargaPulsa {
    public void info() {
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}
```

[Package - hargaToken]

```
public class hargaToken {
    public void info() {
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
        System.out.println("Rp. 500000");
    }
}
```

[Package - main]

```
package com.rio.projectmodul04.Guided2.main;

import com.rio.projectmodul04.Guided2.hargapulsa.hargaPulsa;
import com.rio.projectmodul04.Guided2.hargatoken.hargaToken;

public class main {
  public static void main(String[] args) {

      hargaPulsa pulsa = new hargaPulsa();
      pulsa.info();

      hargaToken token = new hargaToken();
      token.info();
   }
}
```

Output :

```
-----------------------< com.Rio:projectmodul04 >-----------------------
] Building projectmodul04 1.0-SNAPSHOT
   from pom.xml
--------------------------------[ jar ]---------------------------------

] --- resources:3.4.0:resources (default-resources) @ projectmodul04 ---
· skip non existing resourceDirectory C:\Semester 5\pbo file\PBO\templat repo\praktikum-pbo-ps1ds-05-reg-01\modul-0

] --- compiler:3.15.0:compile (default-compile) @ projectmodul04 ---
Recompiling the module because of changed source code.
· Compiling 6 source files with javac [debug release 26] to target\classes

] --- exec:3.6.3:exec (default-cli) @ projectmodul04 ---
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
· Rp. 500000
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

````
Kamu sedang membangun program kecil untuk mencatat dan
memeriksa kualitas dataset sebelum dianalisis, seperti tahap
awal pengecekan *missing value*. Setiap dataset punya nama,
jumlah baris, jumlah kolom, dan jumlah sel yang kosong
(*missing*).

## Petunjuk Umum

Buat **3 package** dengan struktur berikut:

```
Source Packages
├── model
│   └── Dataset.java
├── laporan
│   └── LaporanDataset.java
└── main
    └── Main.java
```

### 1. Class `Dataset` (package `model`)

**Atribut** (implementasikan enkapsulasi):

| Atribut | Tipe | Keterangan |
|---|---|---|
| `nama` | `String` | Nama dataset |
| `jumlahBaris` | `int` | Banyak baris (observasi) |
| `jumlahKolom` | `int` | Banyak kolom (fitur) |
| `jumlahMissing` | `int` | Banyak sel yang kosong |

**Ketentuan tambahan:**

- Buat **konstanta** `BATAS_MISSING` bernilai `5.0` (persen).
Dataset dengan missing diatas batas ini dianggap perlu
dibersihkan.
- Buat **variabel milik class** yang menghitung total objek
`Dataset` yang telah dibuat, beserta **method** untuk ambil
nilainya tanpa perlu membuat objek.

**Constructor** (ada 3, overloading):
1. Tanpa parameter
````

2. Hanya parameter `nama`
3. Lengkap: `nama`, `jumlahBaris`, `jumlahKolom`, `jumlahMissing`

Setiap constructor harus menambah hitungan total dataset.

Method:
- Getter dan setter untuk semua atribut.
- Pada setter `jumlahBaris`, `jumlahKolom`, dan `jumlahMissing`, buat agar nilai tidak berubah.
- `getPersentaseMissing()` mengembalikan `double`: persentase sel kosong dibanding total sel (`baris × kolom`). Jika total sel = 0, kembalikan `0`.
- `perluDibersihkan()` mengembalikan `boolean`: `true` jika persentase missing melebihi `BATAS_MISSING`.

2. Class `LaporanDataset` (package `laporan`)

Buat method `cetak` yang menerima satu objek `Dataset` dan menampilkan laporannya dengan format:

```
=== Laporan Dataset ===
Nama         : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing      : 866 sel (8.10%)
Status       : Perlu dibersihkan
```

Status berisi `Perlu dibersihkan` atau `Bersih` sesuai hasil dari `perluDibersihkan()`.

3. Class `Main` (package `main`)

Buat tiga objek, masing-masing dengan constructor berbeda:

Objek	Cara pembuatan	Data
1	Constructor tanpa parameter, lalu isi memakai setter	Titanic, 891 baris, 12 kolom, 866 missing
2	Constructor 1 parameter	Wine Quality
3	Constructor lengkap	Iris, 150 baris, 5 kolom, 0 missing

Simpan ketiganya dalam **array**, cetak laporannya dengan perulangan, lalu tampilkan di akhir:

```
Total dataset dibuat : 3
```

[LaporanDataset]

```
import java.util.Locale;
import com.rio.projectmodul04.Unguided.model.Dataset;

public class LaporanDataset {
    public static void cetak(Dataset d) {
        String status = d.perluDibersihkan() ? "Perlu dibersihkan" : "Bersih";

        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama         : " + d.getNama());
        System.out.println("Jumlah Baris : " + d.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + d.getJumlahKolom());
        System.out.println(String.format(Locale.US, "Missing      : %d sel
(%.2f%%)",
                d.getJumlahMissing(), d.getPersentaseMissing()));
        System.out.println("Status       : " + status);
    }
}
```

[Main]

```
import com.rio.projectmodul04.Unguided.laporan.LaporanDataset;
import com.rio.projectmodul04.Unguided.model.Dataset;

public class Main {
    public static void main(String[] args) {
        // Objek 1: constructor tanpa parameter, lalu isi dengan setter
        Dataset d1 = new Dataset();
        d1.setNama("Titanic");
        d1.setJumlahBaris(891);
        d1.setJumlahKolom(12);
        d1.setJumlahMissing(866);

        // Objek 2: constructor 1 parameter
        Dataset d2 = new Dataset("Wine Quality");

        // Objek 3: constructor lengkap
        Dataset d3 = new Dataset("Iris", 150, 5, 0);

        // Simpan ketiganya dalam array
        Dataset[] daftar = {d1, d2, d3};

        // Cetak laporan dengan perulangan
        for (Dataset d : daftar) {
            LaporanDataset.cetak(d);
            System.out.println();
        }

        System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
    }
}
```

[Dataset]

```
public class Dataset {
    // Konstanta: batas persentase missing (dalam persen)
    public static final double BATAS_MISSING = 5.0;

    // Variabel milik class: total objek Dataset yang telah dibuat
    private static int totalDataset = 0;

    // Atribut (enkapsulasi: private)
    private String nama;
    private int jumlahBaris;
    private int jumlahKolom;
    private int jumlahMissing;

    // Constructor 1: tanpa parameter
    public Dataset() {
        this("Tanpa Nama", 0, 0, 0);
    }

    // Constructor 2: hanya parameter 'nama'
    public Dataset(String nama) {
        this(nama, 0, 0, 0);
    }

    // Constructor 3: lengkap: 'nama','jumlahBaris','jumlahKolom','jumlahMissing'
    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
jumlahMissing) {
        this.nama = nama;
        setJumlahBaris(jumlahBaris);
        setJumlahKolom(jumlahKolom);
        setJumlahMissing(jumlahMissing);
        totalDataset++; // setiap objek yang dibuat menambah hitungan
    }

    // Method static untuk mengambil total tanpa membuat objek
    public static int getTotalDataset() {
        return totalDataset;
    }

    // Getter
    public String getNama() {
        return nama;
    }

    public int getJumlahBaris() {
        return jumlahBaris;
    }

    public int getJumlahKolom() {
```

```
        return jumlahKolom;
    }

    public int getJumlahMissing() {
        return jumlahMissing;
    }

    // Setter
    public void setNama(String nama) {
        this.nama = nama;
    }

    // Nilai negatif ditolak: nilai lama tidak berubah
    public void setJumlahBaris(int jumlahBaris) {
        if (jumlahBaris >= 0) {
            this.jumlahBaris = jumlahBaris;
        }
    }

    public void setJumlahKolom(int jumlahKolom) {
        if (jumlahKolom >= 0) {
            this.jumlahKolom = jumlahKolom;
        }
    }

    public void setJumlahMissing(int jumlahMissing) {
        if (jumlahMissing >= 0) {
            this.jumlahMissing = jumlahMissing;
        }
    }

    // Persentase sel kosong terhadap total sel (baris x kolom)
    public double getPersentaseMissing() {
        long totalSel = (long) jumlahBaris * jumlahKolom;
        if (totalSel == 0) {
            return 0;
        }
        return (double) jumlahMissing / totalSel * 100;
    }

    // true jika persentase missing melebihi batas
    public boolean perluDibersihkan() {
        return getPersentaseMissing() > BATAS_MISSING;
    }
}
```

Output :

```
--------------------------------[ jar ]---------------------------------

] --- resources:3.4.0:resources (default-resources) @ projectmodul04 ---
skip non existing resourceDirectory C:\Semester 5\pbo file\PBO\templat repo\praktikum-pbo-ps1ds-05-reg-01\modul-04-enka

] --- compiler:3.15.0:compile (default-compile) @ projectmodul04 ---
Recompiling the module because of changed source code.
Compiling 9 source files with javac [debug release 26] to target\classes

] --- exec:3.6.3:exec (default-cli) @ projectmodul04 ---
=== Laporan Dataset ===
Nama         : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing      : 866 sel (8.10%)
Status       : Perlu dibersihkan

=== Laporan Dataset ===
Nama         : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing      : 0 sel (0.00%)
Status       : Bersih

=== Laporan Dataset ===
Nama         : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing      : 0 sel (0.00%)
Status       : Bersih

Total dataset dibuat : 3
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Program pada tugas unguided ini dirancang untuk menerapkan konsep Pemrograman Berorientasi Objek (PBO), khususnya Enkapsulasi, Constructor, Modifier, dan Package, dalam studi kasus pengelolaan serta pengecekan kualitas dataset. Program ini dibagi menjadi tiga package terpisah agar struktur kode modular, rapi, dan mudah dikelola. Package model berisi kelas Dataset yang menerapkan prinsip *data hiding* melalui atribut *private* seperti nama, jumlahBaris, jumlahKolom, dan jumlahMissing. Akses dan modifikasi terhadap atribut tersebut diatur menggunakan metode *getter* dan *setter* yang dilengkapi validasi untuk menolak nilai negatif, serta didukung oleh konstanta BATAS_MISSING dan variabel *static* totalDataset untuk melacak jumlah objek yang tercipta.

D. Kesimpulan

Berdasarkan praktikum Modul 4 mengenai Enkapsulasi, Constructor, dan Package, dapat disimpulkan bahwa penerapan konsep *data hiding* melalui penggunaan access modifier *private* pada atribut kelas serta penyediaan metode *getter* dan *setter* sangat efektif untuk menjaga keamanan dan integritas data. Selain itu, pemanfaatan modifier seperti *static* dan *final* serta penerapan *constructor overloading* memungkinkan proses inisialisasi objek menjadi jauh lebih fleksibel, terstruktur, dan efisien sesuai dengan berbagai skenario kebutuhan parameter yang diinginkan.